
GORGO: Online Tuning for Cross-Region Network-Aware LLM Serving

Anonymous Author(s)

Affiliation

Address

email

Abstract

In cross-region LLM serving, time-to-first-token (TTFT) is highly sensitive to per-request routing decisions. KV-cache reuse across replicas can eliminate much of the prefill computation on multi-turn workloads, but realizing this potential requires jointly balancing cache locality against replica load and network cost. Additionally, public datasets to evaluate these policies do not represent long-context, high prefix-reuse workloads: LMSYS and WildChat average 2 and 2.5 turns per conversation, 0.5 and 2.9k tokens/request, and 9 and 29% cross-user reuse, respectively. To capture the setting where a majority of requests contain reusable prefixes, we evaluate common routing policies such as least-load, prefix-aware, session-affinity, and others on traces from a production inference endpoint featuring multi-turn conversations (~ 82 requests/user), $7\text{--}45\times$ longer prompts (21k avg tokens), and $2\text{--}6\times$ higher KV-cache reuse (55%). We evaluate policies on a multi-region GPU cluster where cross-region network latency ranges from 26ms to 466ms across replicas to test GORGO, a policy that jointly optimizes cache reuse, network latency, and queue depth with weights optimized online to minimize p95 TTFT with no offline profiling. The GORGO family achieves the lowest TTFT p50 and p95 across both a tuning window and a held-out evaluation window against all other load-balancing policies; in the nighttime windows the p99 differences sit at the per-window noise floor, highlighting the impact of online joint optimization. The code is available at <https://anonymous.4open.science/r/GORGO-D25A>.

1 Introduction

Modern LLM chat services serve interactive workloads where perceived latency is dominated by time-to-first-token (TTFT). On a single replica, TTFT is the sum of three terms: (i) the network round-trip from the client proxy to the replica, (ii) queueing delay behind in-flight requests, and (iii) prefill computation for the input prompt. Inference engines such as SGLang [Zheng et al., 2024b] and vLLM [Kwon et al., 2023] expose radix-tree prefix caches that let an incoming request skip prefill for any token prefix already cached on the handling replica. When prompts are long and prefixes are widely reused, this saving dominates: a 20,000-token prompt that hits a 90% prefix match has only $\sim 2,000$ tokens to prefill, reducing TTFT by an order of magnitude.

The KV-cache reuse opportunity is contingent on routing. If a request is sent to a replica that has not seen its prefix, the saving is forfeit. If many requests pile onto the same warm replica, that replica becomes a queueing bottleneck and the cache hit no longer pays for itself. If the closest replica with a warm cache is on the other side of the world, the network round-trip can swallow what the cache hit was supposed to save. The routing layer therefore has to balance three terms simultaneously: network distance to the replica, current load, and reusable cache state for the incoming prompt.

Standard heuristics such as least-load, prefix-aware, and session-affinity routing optimize one of these signals and ignore the others. For workloads with weak prefix reuse and uniform replica latencies, these heuristics are standard practice. However, interactive long-context traffic breaks both assumptions: production chat traces concentrate most tokens in a small set of returning users with substantial prefix overlap, and replicas placed in different regions have round-trip times that differ by an order of magnitude. The right routing target for a given request can change minute-to-minute as load shifts, and weighting the three terms with static rules leaves performance on the table.

Two further factors complicate empirical evaluation. First, the public chat datasets used in prior LLM-serving evaluations underrepresent long-context multi-turn traffic with reusable prefixes. LMSYS-Chat-1M [Zheng et al., 2024a] and WildChat-4.8M [Zhao et al., 2024] average 470 and 2,900 tokens per request, with intra-user prefix reuse below 10%. In order to understand the effects of routing as these factors scale, we evaluate on a production endpoint with 21,000 tokens per request on average and 53.7% intra-user prefix reuse—a regime where joint cache–network–queue trade-offs dominate TTFT. Second, evaluation methodology matters: a routing policy that wins p50 by herding traffic onto a single warm replica typically loses tail latency once that replica saturates. Reporting only one percentile hides this trade-off.

Contributions.

- A workload characterization (§2) placing three datasets on the same prefix-reuse and request-length axes. The production ArtChat-411K trace has $7\text{--}45\times$ longer prompts and $2\text{--}6\times$ higher token-weighted reuse than LMSYS-Chat-1M and WildChat-4.8M.
- A routing cost model and three variants collectively called GORGO (§3), which score each replica by an additive combination of network latency, estimated prefill cost on uncached tokens, and estimated decode cost on queued tokens. Variants differ in how scalar hyperparameters are set. The first variant is a static configuration (`gorgo-static`), the second is a per-replica online fit (`gorgo-autotune`), and the third is a Gaussian (1+1) evolution strategy hill climb that directly minimizes p95 TTFT (`gorgo-hillclimb`).
- An experimental setup (§4) running each of eight policies on its own dedicated three-replica fleet of L40S SGLang servers in Asia, Europe, and the US East coast. Policies are benchmarked in parallel to normalize the variability of cross-region RTT, which spans 26–466 ms.
- Empirical results (§5) across three 30-minute production-trace windows. The GORGO family achieves the lowest TTFT p50 and p95 on the tuning and both held-out windows against the baseline policies, and sweeps all three percentiles in the midday window; the nighttime p99 differences sit at the noise floor (§6).

2 Workload Characterization

Routing policies that perform well on short, low-reuse prompts may perform differently on long, high-reuse prompts because the prefill term in TTFT scales with uncached input tokens. We measure three datasets along the same axes: request length, conversational structure, and token-weighted prefix reuse.

Datasets. **LMSYS-Chat-1M** [Zheng et al., 2024a] is a dataset of one million chatbot conversations from a publicly hosted demo of a Vicuna model and the ChatbotArena website. **WildChat-4.8M** [Zhao et al., 2024] is a corpus of 3.2M ChatGPT-proxy conversations grouped by client IP. Our **ArtChat-411K trace** contains chat completion requests from a production GLM 5.1 model endpoint hosted by an industry company [GLM Team, 2024]. It contains 411,169 requests across 4,984 distinct users, yielding ~ 82 requests per user on average. We used the request metadata for real-world timestamps and tokenized request bodies for reuse calculation.

We measure prefix reuse with a token-weighted prefix trie that ingests each request’s tokenized message sequence in arrival order, partitioned by user where the dataset carries user identity. The trie reports *intra-user savings* (fraction of tokens matching a prefix from the same user’s prior requests), *cross-user savings* (additional fraction from pooling across users), and *global savings* (sum). LMSYS does not carry user identity, so we report only the global figure.

Table 1: Workload characterization. Token counts use the same byte-pair tokenizer in all rows. Reuse percentages are token-weighted prefix-trie savings as described in §2.

Dataset	Total reqs	Users	Avg input tokens	Intra-user reuse	Global reuse
LMSYS-Chat-1M	1,000,000	—	467	—	9.0%
WildChat-4.8M	3,199,860	1,833,730	2,925	5.3%	34.4%
ArtChat-411K	411,169	4,984	21,043	53.7%	55.3%

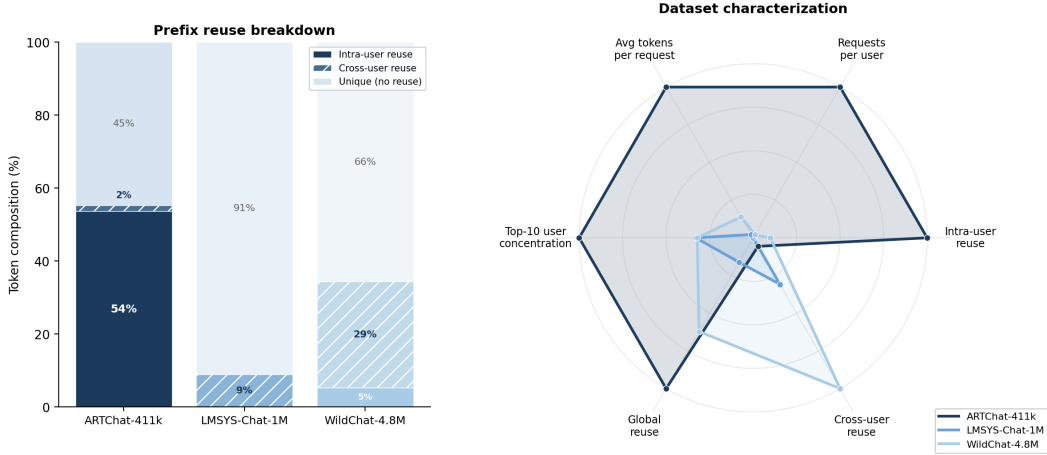


Figure 1: *Left*: Token composition per dataset. ArtChat-411K’s reuse is overwhelmingly intra-user (54%), reflecting multi-turn dialogue; WildChat’s is predominantly cross-user (29%, shared templates); LMSYS has minimal reuse (9%, all cross-user). *Right*: Radar chart with six axes normalized to the dataset maximum. ArtChat-411K dominates on every axis relevant to routing: long prompts, high multi-turn density, concentrated users, and strong intra-user prefix reuse.

Three contrasts stand out (Table 1, Figure 1). *Request length*: ArtChat-411K prompts are $45\times$ longer than LMSYS and $7\times$ longer than WildChat. At typical rates, prefill on a 21,000-token uncached prompt takes on the order of seconds.

User concentration: ArtChat-411K has 82 requests per user on average; WildChat has 1.7; LMSYS does not carry user identifiers, so intra-user statistics cannot be computed. Intra-user reuse depends on returning users, which the public sets simply do not have. *Reuse magnitude*: 53.7% of ArtChat-411K tokens are part of a prefix seen earlier from the same user (5.3% for WildChat, essentially zero for LMSYS). The bulk of WildChat’s 34.4% global reuse comes from cross-user shared structure (chat templates, viral prompts), not sustained per-user dialogue. Thus, for routing, LMSYS and WildChat live in a regime where prefix-aware policies have little to win. On ArtChat-411K, a strategy that lands a user on a replica that has seen their conversation can save more than half of the prefill.

Experimental windows. We tune on a high-diversity 30-minute window from ArtChat-411K traffic on April 5th, 2026: 16:15–16:45 UTC (W1). We then freeze the learned parameters and evaluate on two non-consecutive, held-out windows drawn from later days and time-of-day regimes: April 6th, 15:05–15:35 UTC (W2a), and April 7th, 19:45–20:15 UTC (W2b). This split tests whether parameters learned from one high-diversity window generalize to later traffic rather than simply carrying over to an adjacent interval. The two evaluation windows were selected from the 7-day trace for high user diversity and multi-turn density while avoiding single-user-dominated bursts. Traces are replayed at original arrival speed and pre-filtered to 24,000 input tokens. Output is capped at 128 tokens to prevent decode from dominating latency.

3 GORG0

3.1 TTFT Decomposition

Let a request r with token sequence x_r be routed to replica i . Its TTFT decomposes as

$$\text{TTFT}(r, i) = \text{rtt}(i) + \text{qt}(i, t_r) + \text{pt}(i) \cdot |x_r \setminus c_i(t_r)| + \varepsilon, \quad (1)$$

where $\text{rtt}(u)$ is the round-trip from the routing proxy to replica i ; $\text{qt}(u, t_i)$ is queueing delay; $\text{pt}(i)$ is the per-token prefill rate; $c_i(t)$ is the set of token sequences cached on i at arrival time t_r , so $|x_r \setminus c_i(t_r)|$ is the number of uncached tokens. Cross-region $\text{rtt}(i)$ ranges from 26 ms to 466 ms in our cluster; the prefill term can be the largest and most routing-sensitive term at typical rates of 0.1–1 ms per uncached token on a 21,000-token prompt.

3.2 Cost Model

A GORG0 replica score is a weighted sum of the three controllable terms in Equation 1:

$$\text{score}(i, r) = w_{\text{rtt}} \cdot \text{rtt}_i + (|x_r| - |x_r \cap c_i|) + w_{\text{queue}} \cdot \ell_i(t_r). \quad (2)$$

The proxy sends r to the replica with the minimum score. Three signals feed the score: (i) rtt_i from the Exponential Weighted Moving Average (EWMA) of a lightweight network probe decoupled from the metrics scrape; (ii) $|x_r \cap c_i|$ from the proxy’s local radix trie of cached prefixes per replica, refreshed on every request completion; and (iii) the count of tokens dispatched to each replica for all requests without a response, $\ell_i(t_r)$. The two free parameters are the RTT weight w_{rtt} and the queue weight w_{queue} ; the own-prefill term is fixed as the unit of score.

3.3 Three Variants of Weight Selection

gorgo-static fixes t_{prefill} and α before the run and never adjusts them. In W1 we use manually chosen starter values ($\alpha = 0.06$, $t_{\text{prefill}} = 0.07$). In W2, **gorgo-static** uses the global hyperparameter values from **gorgo-hillclimb** at the end of W1 and deploys them as equivalent, per-replica weights. ($t_{\text{prefill}} \approx 0.983$, $\alpha \approx 4.4 \times 10^{-4}$) and never adjusts them again. This isolates the contribution of the cost model independent of online adaptation.

gorgo-autotune re-fits both parameters every 16 new request samples from the trailing 64-sample window, partitioned by destination replica. The prefill weight t_{prefill} is set to the median observed prefill rate $(\text{TTFT} - \text{rtt})/|x_r \setminus c_u(t_r)|$; the load weight α is set to the median observed decode rate $(\text{totaltime} - \text{TTFT})/n_{\text{output}}$, since queued and in-use tokens drain at the decode rate. It adapts to per-replica hardware drift and RTT shifts, but treats the cost-model weights as identifiable physical rates rather than black-box knobs.

gorgo-hillclimb runs a Gaussian (1+1)-evolution strategy [Rechenberg, 1973] over the same two parameters in log-space, with Rechenberg’s 1/5-success rule for step-size adaptation. The objective is the negative p95 TTFT of the rolling 64-request window. Every 16 new samples the tuner scores the current weights, accepts or rejects the previous candidate against the incumbent, and proposes a new candidate by perturbing each parameter as $\exp(\log(v) + \sigma \mathcal{N}(0, 1))$ with σ adapted according to Rechenberg’s 1/5-success rule. This variant treats the cost-model weights as black-box knobs: it finds whatever values produce the best p95 TTFT under the current arrival distribution, with no offline profiling.

4 Experimental Setup

Hardware and software. Each policy runs on its own dedicated three-replica fleet so routing decisions cannot warm or cool another policy’s caches. Each fleet consists of one 2×L40S SGLang [Zheng et al., 2024b] server (tensor-parallel 2, 96 GB VRAM per replica) in each of Asia (Seoul), Europe (Frankfurt), and US East (Ashburn), serving Qwen3.5-35B-A3B-FP8 [Qwen Team, 2025]. The model’s native context window is 32,768 tokens; we cap workload input at 24,000 tokens to leave KV headroom for in-flight batches. A CPU proxy in US East (Ashburn) hosts the routing policy and replays the trace.

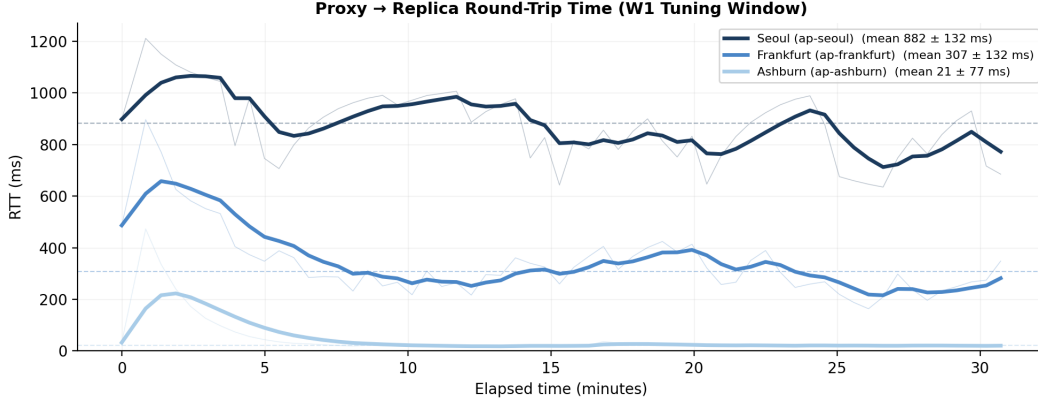


Figure 2: Proxy-to-replica RTT over the W1 tuning window. Bold lines show the EWMA-smoothed signal ($\alpha=0.3$) that GORGO uses for routing; faint lines show raw probe samples. Dashed lines show per-replica means. Seoul exhibits periodic spikes from routing-table churn; Frankfurt is moderately variable; Ashburn is near-constant. The $18\times$ spread motivates network-aware routing.

153 The proxy scrapes Prometheus metrics from each replica every ~ 30 s and runs a separate lightweight
 154 HTTP probe to estimate per-replica RTT, smoothed with an EWMA. The probe is decoupled from
 155 the metrics scrape to avoid polluting the RTT estimate with SGLang handler contention. Before
 156 each run, a homogeneity check issues 16 streaming requests directly to each replica, bypassing the
 157 proxy; runs with worst-to-median TTFT ratio exceeding $3\times$ are flagged. No such run occurred in
 158 these experiments.

159 **Cross-region RTT distribution.** Probed from the US East proxy, EWMA-smoothed RTT to the
 160 US East replica is 26–40 ms, to Frankfurt is 90–130 ms, and to Seoul is 260–466 ms. The Seoul
 161 upper bound reflects routing-table churn during the trace. The overall range of 26–466 ms represents
 162 an $18\times$ spread. Figure 2 shows the full RTT time series over the W1 window.

163 **Baselines, policies, and windows.** We compare the three GORGO variants of §3 (`gorgo-static`,
 164 `gorgo-autotune`, `gorgo-hillclimb`) against five baseline policies. `random` samples a replica
 165 uniformly per request and serves as the lower bound. `least-request` routes to the replica
 166 with the fewest in-flight requests, taking the larger of the proxy’s in-flight view and SGLang’s
 167 scraped running-request count to bridge the ~ 10 s staleness between metrics scrapes. `least-load`
 168 routes to the replica with the lowest aggregate token-weighted load, summing running and queued
 169 requests with used and queued KV tokens; it is heavier-signal than `least-request` but cache-
 170 blind. `prefix-cache` is the longest-prefix-match policy of Preble [Srivatsa et al., 2025] as
 171 packaged in AIBrix [AIBrix Team, 2024]: it picks the replica with the longest cached pre-
 172 fix match and falls back to `least-request` when the running-request imbalance exceeds a soft
 173 threshold. `simple-session-affinity` hashes the first 256 token IDs of the prompt mod-
 174 ulo the number of replicas, maximizing intra-user reuse but providing no load-escape mecha-
 175 nism. We evaluate these six policies in each of two windows; all six fleets start simultane-
 176 ously at the same wall-clock time with the same input trace. W1 supplies `gorgo-hillclimb`
 177 with uniform starting weights (`rtt_weight=1.0`, `prefill_weight=1.0`, `load_weight=1.0`); W2 uses
 178 the weights that `gorgo-hillclimb` learned during W1 (`rtt_weight $\approx$ 0.39`, `prefill_weight $\approx$ 1.88`,
 179 `load_weight $\approx$ 6.38`). `gorgo-static` in W2 deploys those weights without further adjustment. Con-
 180 currency is 64 in-flight requests per proxy.

181 **Metrics.** For each policy and window we report TTFT p50, p95, and p99 over all successful
 182 streaming responses. Traces are pre-filtered to the model’s effective context boundary; all policies
 183 achieve 100% success rate in all windows.

Table 2: ArtChat-411K W1 (tuning window, 00:30–01:00 UTC, April 2nd 2026). TTFT and E2E in seconds. Bold is best in column. $n=2,095$ requests per policy; 100% success rate. gorgo-hillclimb actively explores via (1+1)-ES with $\text{load_weight} \in [0.1, 10.0]$.

Policy	TTFT (s)			E2E (s)			ITL avg (ms)	Decode tok/s	Succ. %
	p50	p95	p99	p50	p95	p99			
gorgo-hillclimb	0.180	1.010	2.660	1.42	2.14	2.95	10.1	105	100
simple-session-affinity	0.194	1.185	6.757	1.48	2.98	3.47	10.1	106	100
least-request	0.197	1.179	2.062	1.24	2.24	3.07	8.1	125	100
least-load	0.271	2.158	8.750	1.40	3.30	2.92	9.1	116	100
prefix-cache	0.283	1.305	2.210	1.31	2.74	3.32	9.6	110	100
random	0.292	1.192	1.996	1.27	2.38	3.24	8.5	113	100

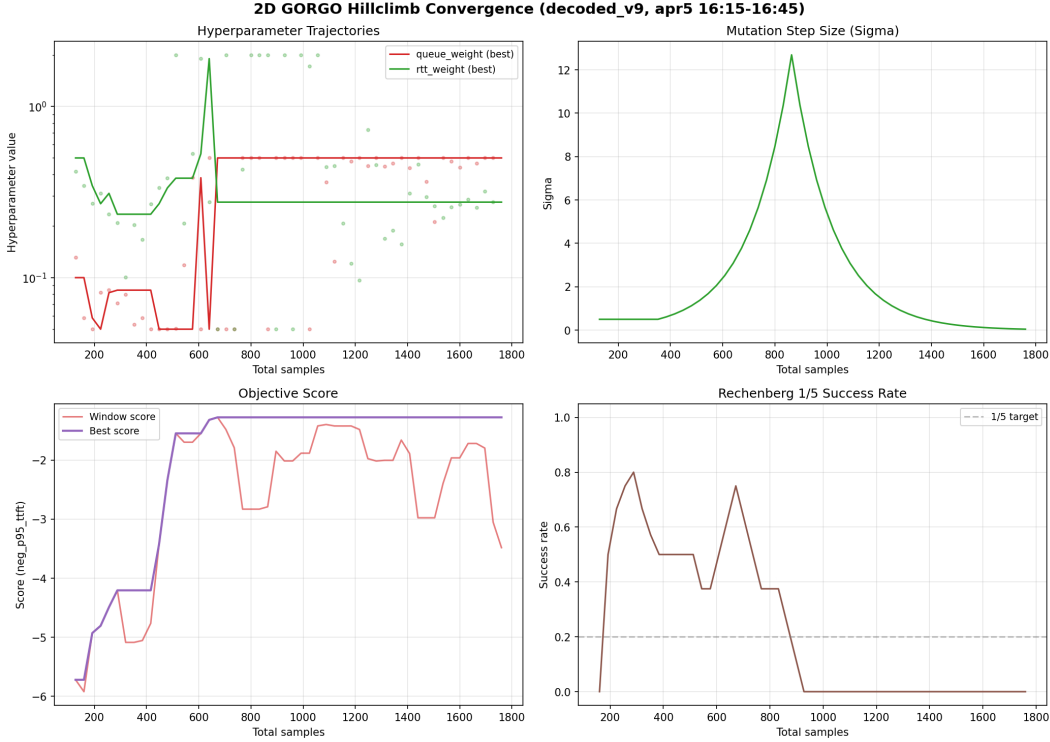


Figure 3: (1+1)-ES convergence over the W1 tuning window (p95 objective). *Top-left*: weight trajectories— w_{load} (orange) climbs to 6.38 while w_{rtt} (green) settles at 0.39; dots show ES proposals. *Top-right*: step size σ peaks then decays as the ES converges. *Bottom-left*: best p95 TTFT improves from 3.0 s to 0.5 s. *Bottom-right*: acceptance rate drops toward the 1/5 target.

184 5 Results

185 5.1 p95 TTFT Objective

186 **Tuning window.** In W1 (Table 2, Figure 4), gorgo-hillclimb delivers the lowest p50 (0.180 s,
 187 7.4% ahead of simple-session-affinity), p95 (1.010 s, 14.4% ahead of least-request), and
 188 E2E p95 (2.14 s, 4.7% ahead of least-request). The ES exploration tax inflates p99 (2.660 s vs.
 189 random’s 1.996 s); this cost disappears in the evaluation windows where gorgo-static deploys
 190 frozen weights.

191 Starting from $\text{rtt_weight}=1.0$, $\text{prefill_weight}=1.0$, $\text{load_weight}=1.0$, the ES converged to
 192 $\text{rtt_weight}\approx 0.39$, $\text{prefill_weight}\approx 1.88$, $\text{load_weight}\approx 6.38$ over the 30-minute window. The high
 193 load_weight means the ES discovered that actively avoiding loaded replicas is critical at $c=64$ con-
 194 currency, even at the cost of occasionally routing to a cache-cold replica.

Table 3: ArtChat-411K W2a (held-out nighttime evaluation, 01:00–01:30 UTC, April 2nd 2026). gorgo-static deploys the W1-learned weights without adjustment. Bold is best in column. $n=2,207$ requests per policy; 100% success rate.

Policy	TTFT (s)			E2E (s)			ITL avg (ms)	Decode tok/s	Succ. %
	p50	p95	p99	p50	p95	p99			
gorgo-static	0.186	0.896	1.844	1.23	1.99	2.89	8.5	121	100
prefix-cache	0.190	1.041	2.011	1.35	2.54	3.16	9.4	113	100
simple-session-affinity	0.201	0.981	1.785	1.54	2.64	3.47	10.6	102	100
least-request	0.260	1.131	1.758	1.23	2.16	2.90	8.1	127	100
least-load	0.276	1.144	1.808	1.42	2.43	3.28	9.5	112	100
random	0.292	1.280	2.481	1.30	2.49	4.38	8.5	123	100

Table 4: ArtChat-411K W2b (held-out midday diurnal evaluation, 12:30–13:00 UTC, April 2nd 2026). Different time-of-day from W1; 47% more requests. Bold is best in column. $n=3,071$ requests per policy; 100% success rate.

Policy	TTFT (s)			E2E (s)			ITL avg (ms)	Decode tok/s	Succ. %
	p50	p95	p99	p50	p95	p99			
gorgo-static	0.195	1.136	2.060	1.29	2.46	3.24	8.8	119	100
simple-session-affinity	0.234	1.519	2.539	1.69	3.78	6.70	12.1	96	100
prefix-cache	0.280	1.558	2.491	1.55	3.59	7.14	11.1	103	100
random	0.292	1.518	2.457	1.37	3.01	4.53	9.3	115	100
least-request	0.294	1.297	2.064	1.36	2.60	3.48	9.0	116	100
least-load	0.297	2.030	3.765	1.73	4.00	5.99	11.7	96	100

Held-out evaluation. In W2a (Table 3), gorgo-static—deploying the W1-learned weights without adjustment—achieves the best TTFT p50 (0.186 s, 2.1% gap over prefix-cache), TTFT p95 (0.896 s, 8.7% gap over simple-session-affinity at 0.981 s), and sweeps E2E: p50 (1.23 s), p95 (1.99 s, 7.9% gap over least-request at 2.16 s), and p99 (2.89 s). The frozen W1 weights generalize to the held-out nighttime trace with improved margins—the absence of ES exploration removes the tail excursions that inflate p99 during tuning.

In the midday diurnal evaluation (Table 4), gorgo-static sweeps every metric: TTFT p50 (0.195 s, 16.4% gap over simple-session-affinity), TTFT p95 (1.136 s, 12.4% gap over least-request), TTFT p99 (2.060 s), E2E p50 (1.29 s, 5.9% gap), E2E p95 (2.46 s, 5.6% gap), E2E p99 (3.24 s, 6.7% gap), ITL (8.8 ms), and decode throughput (119 tok/s). The advantage grows under the heavier midday load: gorgo-static’s TTFT p95 degrades 27% from the nighttime eval while simple-session-affinity degrades 55%, because the learned $\text{load_weight}=6.38$ actively rebalances traffic while hash-based routing cannot adapt.

5.2 p50 TTFT Objective

To test whether the learned weights depend on the objective percentile, we re-run W1 tuning with a p50 TTFT objective (same trace, same hyperparameter ranges).

The p50-tuned ES converged to $w_{\text{rtt}}=1.21$, $w_{\text{prefill}}=0.52$, $w_{\text{load}}=1.69$ —a qualitatively different operating point from the p95 run. gorgo-hillclimb achieves 0.163 s p50 TTFT (16.8% gap over simple-session-affinity) and 0.943 s p95 TTFT (4.5% gap). However, the RTT-first routing concentrates traffic on the nearest replica, degrading E2E: p95 E2E is 2.40 s, 14.1% worse than least-request (2.10 s). This tradeoff—better median TTFT at the cost of E2E—is the mirror image of the p95 policy, which wins both.

Held-out evaluation. In W2a, gorgo-static with p50-tuned weights sweeps TTFT: p50 (0.157 s, 23.8% gap over simple-session-affinity), p95 (0.961 s, 3.4% gap), and p99 (1.776 s). E2E p95 is 2.42 s—6.6% worse than least-request (2.27 s), consistent with the RTT-first routing tradeoff observed in W1.

In W2b (midday), gorgo-static with p50-tuned weights wins TTFT p50 (0.188 s, 21.7% gap), p95 (1.184 s, 12.0% gap over least-request), and p99 (1.901 s, 12.3% gap). E2E p95 is 2.92 s,

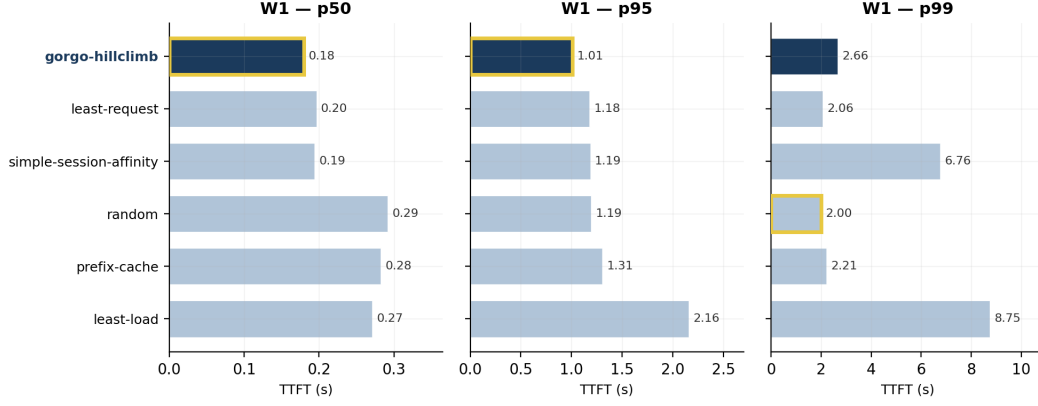


Figure 4: W1 TTFT broken out by percentile (sorted by p95, worst at top). gorgo-hillclimb (gold outline) wins p50 and p95 during the tuning window; random wins p99 (the ES exploration tax inflates the tail).

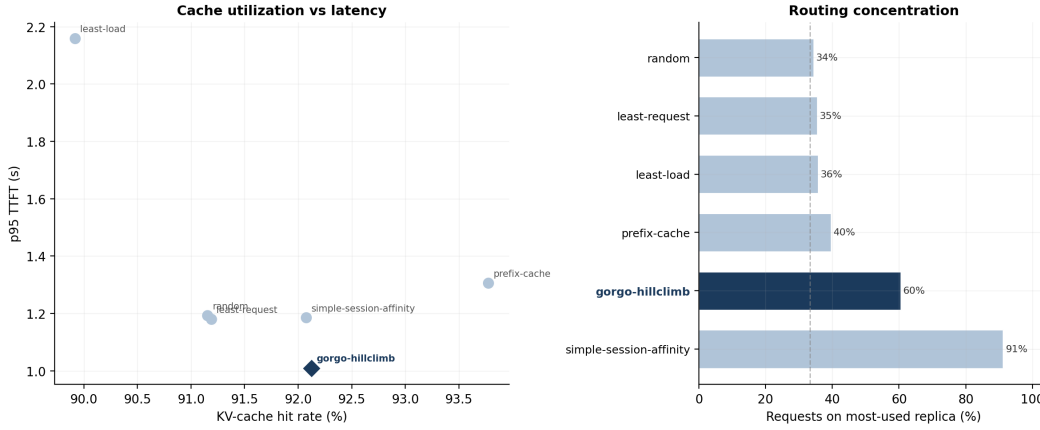


Figure 5: *Left*: KV-cache hit rate vs. p95 TTFT on W2. Bottom-right is ideal (high cache, low latency). gorgo-static and gorgo-hillclimb achieve both; simple-session-affinity achieves the highest cache hit rate but the worst p95 because it cannot rebalance load. *Right*: routing concentration (share of requests on the most-used replica). The dashed line marks uniform (33%). GORG variants concentrate moderately on cache-warm replicas without the pathological imbalance of simple-session-affinity.

223 8.9% worse than least-request (2.68 s). The p50-tuned policy achieves stronger TTFT p50 margins
 224 than the p95-tuned policy (21.7% vs. 16.4%) but pays a consistent E2E penalty—the lower
 225 $w_{\text{load}}=1.69$ (vs. 6.38) provides less load balancing under the heavier midday traffic.

226 5.3 Objective Sensitivity: Weight Comparison

227 Table 8 compares the learned weights across objectives.

228 The p95 objective drives the policy toward cache-first routing ($w_{\text{prefill}}=1.88$) with aggressive load
 229 avoidance ($w_{\text{load}}=6.38$), because tail requests are the ones stuck behind queues on cache-cold repli-
 230 cas. The p50 objective drives toward RTT-first routing ($w_{\text{rtt}}=1.21$, $3.1 \times$ larger), because the typical
 231 request is short enough that network latency dominates over cache effects. The p50-tuned policy
 232 achieves 0.163 s p50 TTFT (vs. 0.180 s for p95-tuned), a 9% improvement on the metric it opti-
 233 mizes.

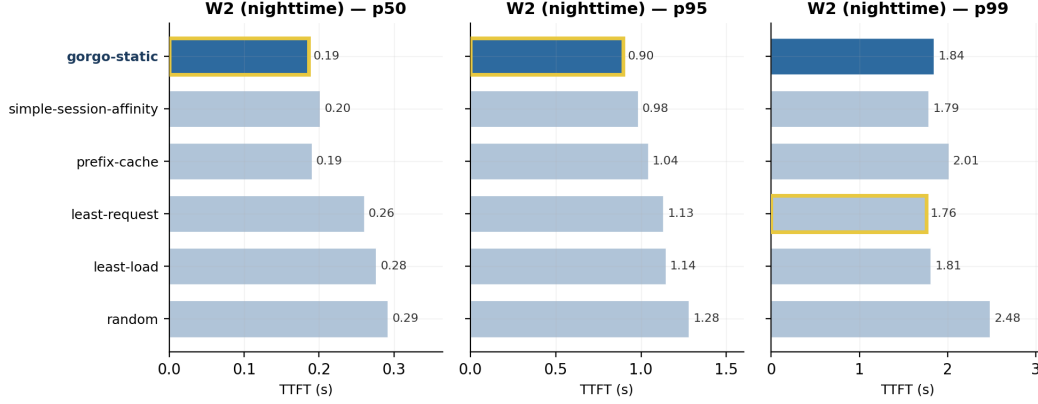


Figure 6: W2a (nighttime held-out) TTFT by percentile. gorgo-static (gold outline) wins p50 and p95; the absence of ES exploration removes the tail excursions seen in W1.

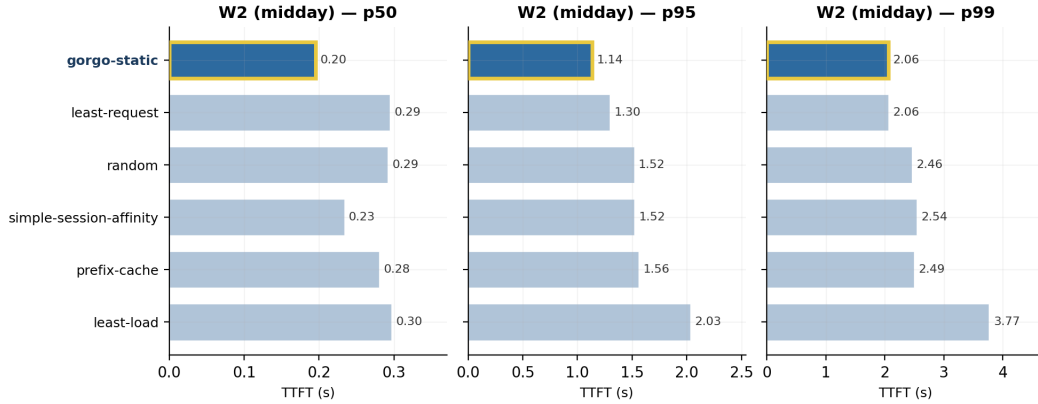


Figure 7: W2b (midday diurnal) TTFT by percentile. gorgo-static (gold outline) sweeps all three percentiles. The advantage over baselines *widens* under heavier midday load.

234 5.4 Why the load term is essential: TTFT-only tuning concentrates load

235 The high learned w_{load} is not incidental — it is what stops the policy from reward-hacking the
 236 TTFT objective. Under continuous batching a saturated replica still admits a new request’s prefill
 237 and emits its first token roughly on time, so *TTFT is largely load-insensitive*; the decode backlog
 238 instead dilutes the running batch and inflates end-to-end (E2E) latency. A policy that optimizes
 239 TTFT alone therefore sees no penalty for herding every request onto the single warmest replica.
 240 Left unconstrained, the ES finds exactly this degenerate solution: it drives w_{load} to zero, routes
 241 100% of traffic to one replica, and posts the *best* TTFT on the nighttime trace, while E2E p95 on
 242 the heavier midday trace explodes to 12.58s—4.8 $\times$ worse than least-request (Appendix C).
 243 Constraining the load weight (which the ES then rails toward $w_{\text{load}} \approx 6.38$) trades $\sim 3\%$ TTFT p95
 244 for $\sim 5\times$ better E2E p95. The lesson is that **E2E is the honest objective on a saturating fleet**:
 245 TTFT is the one axis that does not price the decode backlog, so optimizing it alone optimizes for
 246 hiding the cost.

247 This tension is an artifact of co-locating prefill and decode on one replica. Under prefill/decode
 248 disaggregation [Zhong et al., 2024, Patel et al., 2024], a request prefills on one pool and decodes
 249 on another, decoupling the decode dilution from the prefill routing decision GORGO makes; the
 250 router would then score only TTFT-relevant terms (RTT, uncached prefill, and prefill-pool queue-
 251 ing), and concentrating prefill onto a cache-warm worker would no longer blow up decode E2E.
 252 Characterizing GORGO on a disaggregated fleet is left to future work.

Table 5: ArtChat-411K W1 with p50 TTFT objective (tuning window, 00:30–01:00 UTC, April 2nd 2026). Same trace and ranges as the p95 run. $n=2,095$ requests per policy; 100% success rate.

Policy	TTFT (s)			E2E (s)			Succ. %
	p50	p95	p99	p50	p95	p99	
gorgo-hillclimb	0.163	0.943	2.185	1.45	2.40	3.95	100
simple-session-affinity	0.196	0.988	1.654	1.48	2.61	3.47	100
least-request	0.207	1.102	1.934	1.24	2.10	3.07	100
least-load	0.264	1.079	1.874	1.40	2.38	2.92	100
random	0.274	1.186	2.021	1.27	2.33	3.24	100
prefix-cache	0.287	1.052	1.773	1.31	2.34	3.32	100

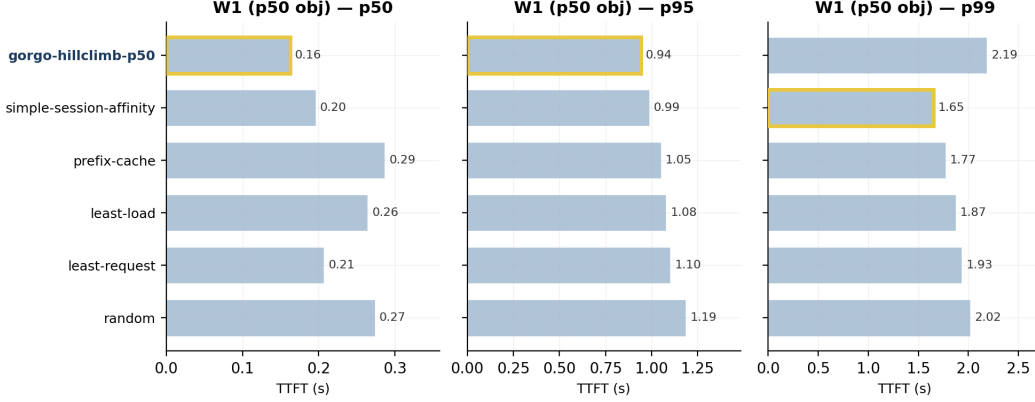


Figure 8: W1 TTFT with p50 objective. gorgo-hillclimb (gold outline) achieves the best p50 (0.163 s) by prioritizing RTT over cache.

6 Limitations

p99 noise floor. Each window has $\approx 2,100$ – $2,200$ successful responses, so the p99 standard error is $\sigma_{\text{TTFT}}/\sqrt{n \cdot 0.01 \cdot 0.99} \approx 0.07$ s. In W2a the two best p99 latencies are **least-request** (1.758 s) and **simple-session-affinity** (1.785 s)—a 0.027 s gap, well inside one SE—with **gorgo-static** at 1.844 s; in W1 the best p99 is random (1.996 s). The nighttime p99 ordering among the top policies is therefore not individually significant in either direction at this sample size, and p50 and p95 carry the comparison. **gorgo-static** does sweep all three percentiles in the heavier midday window (Table 4).

Scope of fleet. Each policy runs on its own dedicated three-replica fleet of identical GPUs. Multi-tenant sharing, heterogeneous hardware, and prefill/decode disaggregation [Zhong et al., 2024, Patel et al., 2024] are not characterized here and would require additional cost terms. The score uses only HTTP-exposed metrics, not scheduler-internal signals [Pan et al., 2025, Yang et al., 2025a] that could tighten the prefill estimate.

Live-tuning instability. Running the (1+1)-ES live can produce a heavy-tailed p99 excursion (Table 4: gorgo-hillclimb p99 inflates to 7.186 s) because the negative-p95 objective does not penalize a single bad-minute proposal fast enough. The freeze-for-production recommendation in §5.1 is therefore a safety property, not just a convenience. Decode dynamics such as the effect of variable length decode and high memory consumption on load may impact live-tuning instability; exploring the impact of higher output token limits would be a direction for future work.

7 Related Work

Prefix caches and reuse. RadixAttention in SGLang [Zheng et al., 2024b] stores per-request KV state in a radix tree; PagedAttention in vLLM [Kwon et al., 2023] enables prefix sharing through

Table 6: p50-objective W2a (held-out nighttime, 01:00–01:30 UTC, April 2nd 2026). gorgo-static deploys the p50-tuned weights ($w_{\text{rtt}}=1.21$, $w_{\text{prefill}}=0.52$, $w_{\text{load}}=1.69$). $n=2,207$ requests per policy; 100% success rate.

Policy	TTFT (s)			E2E (s)			Succ. %
	p50	p95	p99	p50	p95	p99	
gorgo-static	0.157	0.961	1.776	1.42	2.42	3.27	100
simple-session-affinity	0.206	0.995	1.960	1.56	2.67	3.45	100
least-request	0.238	1.180	1.812	1.25	2.27	3.27	100
least-load	0.289	1.298	2.052	1.41	2.57	3.27	100
prefix-cache	0.290	1.160	1.944	1.51	2.51	8.65	100

Table 7: p50-objective W2b (held-out midday diurnal, 12:30–13:00 UTC, April 2nd 2026). $n=3,071$ requests per policy; 100% success rate.

Policy	TTFT (s)			E2E (s)			Succ. %
	p50	p95	p99	p50	p95	p99	
gorgo-static	0.188	1.184	1.901	1.60	2.92	4.83	100
simple-session-affinity	0.240	1.498	2.523	1.71	3.78	8.21	100
random	0.289	1.421	2.331	1.40	2.93	3.97	100
prefix-cache	0.293	1.451	2.973	1.59	3.22	5.29	100
least-request	0.293	1.345	2.167	1.36	2.68	3.83	100
least-load	0.303	1.831	3.144	1.66	3.91	8.36	100

275 paged memory. KVLink [Yang et al., 2025b], ChunkKV [Liu et al., 2025], KVFlow [Pan et al.,
 276 2025], and Learned Prefix Caching [Yang et al., 2025a] improve the single-replica cache but do not
 277 decide which replica serves a request.

278 **Cross-replica routing and cross-region serving.** Preble [Srivatsa et al., 2025] introduces longest-
 279 prefix-match routing across replicas with a load-balance fallback; AIBrix [AIBrix Team, 2024] pack-
 280 ages a production-grade variant of the same design and supplies the baseline we run. Mooncake [Qin
 281 et al., 2024] is a KV-cache-centric architecture and the source of our trace format. SkyServe [Mao
 282 et al., 2025] and SkyWalker [Xia et al., 2025] address cross-region LLM serving at the placement
 283 and spillover layers respectively, but treat per-request routing as out of scope. k-LPM [Dexter et al.,
 284 2025] formulates LLM scheduling under TTFT constraints as NP-hard. DLPM [Cao et al., 2025]
 285 targets fairness with locality. Llumnix [Sun et al., 2024] migrates requests across replicas. Multi-
 286 LLM routers [Wu and Silwal, 2025] address model selection. CacheBlend [Yao et al., 2024] routes
 287 by trie-measured prefix length but does not measure network RTT. DistServe [Zhong et al., 2024]
 288 and Splitwise [Patel et al., 2024] disaggregate prefill from decode. GORGO is the first single-router
 289 policy in this space that scores cache locality, replica load, and wide-area latency in one cost and
 290 derives the cost weights from the deployment’s own per-request TTFT stream, with no engine mod-
 291 ifications.

292 **Online hyperparameter adaptation.** Bayesian and bandit-based methods have been applied to
 293 serving configuration [Wu and Silwal, 2025]. For a 2-dimensional parameter space the (1+1)-ES
 294 [Rechenberg, 1973] provides fast, overhead-free convergence with no surrogate model.

295 8 Conclusion

296 Routing across LLM replicas is a three-signal decision balancing cache locality, replica load, and
 297 wide-area latency, but production heuristics commit to one signal and degrade when that stops being
 298 the limiting factor. We treat the three as terms of an additive per-replica cost and let online TTFT
 299 feedback optimize scaling weights, in place of operator-tuned constants or offline profiling. On
 300 a long-context, high-prefix-reuse production trace the GORGO policy family achieves the lowest
 301 TTFT p50 and p95 across the tuning and held-out windows (and sweeps all three percentiles in
 302 the heavier midday window), with the nighttime p99 ordering between the top policies at the noise
 303 floor. On a short-prompt, low-reuse public trace (Appendix A) the same policy is non-competitive,

Table 8: Learned weights under different objective metrics, same tuning trace and hyperparameter ranges. The ES discovers qualitatively different operating points depending on which percentile it optimizes.

Objective	w_{ttt}	w_{prefill}	w_{load}
neg_p95_ttft	0.39	1.88	6.38
neg_p50_ttft	1.21	0.52	1.69

in agreement with the regime characterization in §2. The advantage is therefore a property of the workload regime as much as of the policy: it scales with how much of TTFT is recoverable through prefill reuse, queue avoidance, or network selection rather than fixed by hardware. The design choices of GORGO transfer to deployments where the workload regime is not known in advance and a dedicated calibration window before each redeploy is not affordable, making it effective in production LLM serving on long context, distributed workloads.

References

- AIBrix Team. AIBrix: An open-source infrastructure for LLM inference at scale. <https://github.com/vllm-project/aibrix>, 2024.
- Shiyi Cao, Yichuan Wang, Ziming Mao, Pin-Lun Hsu, Liangsheng Yin, Tian Xia, Dacheng Li, Shu Liu, Yineng Zhang, Yang Zhou, Ying Sheng, Joseph Gonzalez, and Ion Stoica. Locality-aware fair scheduling in LLM serving. *arXiv preprint arXiv:2501.14312*, 2025.
- Gregory Dexter, Shao Tang, Ata Fatahi, Qingquan Song, Tejas Dharamsi, and Aman Gupta. LLM query scheduling with prefix reuse and latency constraints. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- GLM Team. GLM-4: A family of open multilingual multimodal chat LLMs, 2024.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. ChunkKV: Semantic-preserving KV cache compression for efficient long-context LLM inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. SkyServe: Serving ai models across regions and clouds with spot instances. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025. doi: 10.1145/3689031.3717459.
- Zaifeng Pan, Ajikumar Patel, Zhengding Hu, Yipeng Shen, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2507.07400.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, 2024.
- Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yanzhe Wu, Weimin Zheng, Xinran Chen, Rui Xu, Xingda Yao, Rongxin Wei, Zhiyuan Zhao, Jing Huang, Mingjie Yang, and Jidong Wu. Mooncake: A KVCache-centric disaggregated architecture for LLM serving, 2024.
- Qwen Team. Qwen3 technical report. 2025. Alibaba Cloud.
- Ingo Rechenberg. Evolutionsstrategie: Optimierung technischer systeme nach prinzipien der biologischen evolution. 1973.

343 Vikranth Srivatsa, Sumanth Madhavan, Tian Hu, Sarah Tarun, Yuhao Cheng, Wei Han, Jingyu Yu,
344 Roberto Cristian Rusti, Lifeng Wang, Yiyang Liu, and Hao Zhang. Preble: Efficient distributed
345 prompt scheduling for LLM serving. In *International Conference on Learning Representations*
346 *(ICLR)*, 2025.

347 Biao Sun, Ziming Huang, Hanyu Tang, Chengquan Wang, Cheng Zhang, Yiming Li, Liu Yang,
348 Wencong Zhang, Lin-Wei Wang, Tianhe Wu, Ang Cao, Yongqiang Lin, et al. Llumix: Dynamic
349 scheduling for large language model serving. In *USENIX Symposium on Operating Systems De-*
350 *sign and Implementation (OSDI)*, 2024.

351 Fangzhou Wu and Sandeep Silwal. Efficient training-free online routing for high-volume multi-LLM
352 serving. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

353 Tian Xia, Ziming Mao, Jamison Kerney, Ethan J. Jackson, Zhifei Li, Jiarong Xing, Scott Shenker,
354 and Ion Stoica. SkyWalker: A locality-aware cross-region load balancer for LLM inference. arXiv
355 preprint arXiv:2505.24095, 2025.

356 Dongsheng Yang, Austin Li, Kai Li, and Wyatt Lloyd. Learned prefix caching for efficient LLM
357 inference. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025a.

358 Jingbo Yang, Bairu Hou, Wei Wei, Yujia Bao, and Shiyu Chang. KVLink: Accelerating large
359 language models via efficient KV cache reuse. *arXiv preprint arXiv:2502.16002*, 2025b.

360 Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Peng, Hao Zhang, Ion Stoica, Kurt Keutzer,
361 and Michael W. Mahoney. CacheBlend: Fast large language model serving for RAG with cached
362 knowledge fusion, 2024.

363 Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. WildChat: 1M
364 ChatGPT interaction logs in the wild. In *International Conference on Learning Representations*
365 *(ICLR)*, 2024.

366 Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Tianle Li, Siyuan Zhuang, Zhanghao Wu, Yong-
367 hao Zhuang, Zhuohan Li, Zi Lin, Eric P. Xing, Joseph E. Gonzalez, Ion Stoica, and Hao Zhang.
368 LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. In *International Confer-*
369 *ence on Learning Representations (ICLR)*, 2024a.

370 Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu,
371 Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng.
372 SGLang: Efficient execution of structured language model programs. In *Advances in Neural*
373 *Information Processing Systems (NeurIPS)*, 2024b.

374 Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao
375 Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language
376 model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*,
377 2024.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract claims (i) public datasets underrepresent long-context multi-turn traffic (quantified in §2), (ii) we evaluate routing policies on a production trace (Tables 2, 3, 4), and (iii) **GORG**O achieves the lowest TTFT p50 and p95 in both 30-minute windows, and sweeps all three percentiles in the midday window. We are explicit in the introduction, §5.1, and §6 that “GORG”O refers to a family, that the nighttime p99 ordering is at the noise floor, and that on the daytime stress window gorgo-hillclimb’s p99 inflates and prefix-cache wins p99 instead.

2. Limitations

Question: Does the paper discuss the limitations of the work?

Answer: [Yes]

Justification: Section 6 lists single-trace generalization, the W2 p99 noise floor, the single-tenant assumption, gorgo-autotune instability, hardware homogeneity, the absence of PD-disaggregation, and ES convergence time.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete proof?

Answer: [N/A]

Justification: The paper presents an additive cost model and an online optimizer with no formal theorems.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results?

Answer: [Yes]

Justification: Section 4 specifies model, regions, hardware, fleet topology, concurrency, trace windows, and per-window GORG”O seeding. Section 3 gives Equation 2, the (1+1)-ES configuration ($\sigma_0 = 0.5$, 1/5-rule, hop 16, window 64), and the median-of-rates fit.

5. Open access to data and code

Question: Does the paper provide open access to the data and code?

Answer: [Yes]

Justification: The proxy and policy code are available at <https://anonymous.4open.science/r/GORG-D25A>. The ArtChat-411K trace is not redistributable under its terms of service. The public-dataset characterization (LMSYS, WildChat) is fully reproducible with no proprietary inputs.

6. Experimental setting/details

Question: Does the paper specify all the training and test details?

Answer: [Yes]

Justification: Section 4 specifies model, hardware, regions, concurrency, trace windows, max input tokens, and per-window seeding for adaptive policies. ES configuration is given in Section 3.

7. Experiment statistical significance

Question: Does the paper report error bars or appropriate statistical significance information?

Answer: [Yes]

Justification: Section 6 derives an approximate p99 standard error and shows that the W2a p99 margin between the top policies (least-request and simple-session-affinity) is inside one standard error, while the W1 and W2 p50/p95 wins are several standard errors clear of zero.

8. Experiments compute resources

Question: Does the paper provide sufficient information on compute resources?

Answer: [Yes]

Justification: Each policy runs on a dedicated 3-replica fleet of L40S SGLang servers

(tensor-parallel 2, 96 GB VRAM per replica). Eight policies $\times$ two windows = 16 fleet-runs, each $\sim$ 30 min wall clock, on three regions. Total GPU-hours: $\approx$ 24.

9. Code of ethics

Question: Does the research conform with the NeurIPS Code of Ethics?

Answer: [Yes]

Justification: The work concerns routing infrastructure. The production trace is used under terms that permit research replay; we release only aggregated prefix-trie statistics, not individual requests. No individuals are identifiable from the released aggregates.

10. Broader impacts

Question: Does the paper discuss potential societal impacts?

Answer: [Yes]

Justification: Routing improvements reduce the energy and dollar cost of serving LLMs at fixed quality. Lower-cost serving may accelerate deployment in settings where that is socially harmful; the contributions are infrastructure-level and do not affect model capability.

11. Safeguards

Question: Does the paper describe safeguards for responsible release?

Answer: [N/A]

Justification: We do not release datasets or models. The released code is a routing proxy and policy library with no inherent dual-use beyond LLM-serving infrastructure.

12. Licenses

Question: Are creators or owners of assets properly credited?

Answer: [Yes]

Justification: SGLang, vLLM, AIBrix, LMSYS-Chat-1M, WildChat-4.8M, and Qwen3 are cited and used under their published licenses.

13. New assets

Question: Are new assets introduced in the paper well documented?

Answer: [Yes]

Justification: The proxy, policy library, experiment controller, and trace builder are documented in the repository’s top-level documentation. Spec and manifest files are stored alongside the controller.

14. Crowdsourcing and research with human subjects

Question: Does the paper involve crowdsourcing or human subjects?

Answer: [N/A]

Justification: No human subjects.

15. IRB approvals

Answer: [N/A]

Justification: No human subjects.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the research methodology?

Answer: [N/A]

Justification: LLM serving infrastructure is the object of study. Authors used LLMs for routine writing assistance only.

477 A WildChat replay (regime-sensitivity control)

478 We replay a 30-minute window of WildChat-4.8M [Zhao et al., 2024] on the same $2 \times L40S$ three-
 479 region fleet, with the same eight policies as the ArtChat-411K sweeps. WildChat averages 2,925
 480 input tokens per request and 5.3% intra-user prefix reuse—an order of magnitude shorter than
 481 ArtChat-411K and roughly a tenth the reuse—putting it well outside the regime characterized in
 482 §2.

Table 9: WildChat-4.8M replay: 30-minute window, $c=32$. TTFT in seconds. Bold is best in column. Compared to the ArtChat-411K results, GORGO variants are not competitive: gorgo-static and gorgo-autotune are the worst two policies on p95.

Policy	Completed	p50	p95	p99	max	Succ. %
simple-session-affinity	20,447	0.416	1.025	1.712	7.75	99.6
least-request	20,517	0.480	1.036	1.731	110.45	100.0
random	20,436	0.416	1.077	1.796	15.82	99.6
prefix-cache	20,504	0.497	1.186	2.588	10.63	99.9
least-load	20,484	0.532	1.217	2.535	8.28	99.8
gorgo-hillclimb	20,473	0.541	1.325	2.654	7.05	99.8
gorgo-static	20,462	0.709	1.932	3.969	60.18	99.7
gorgo-autotune	20,451	0.541	3.583	6.438	12.55	99.7

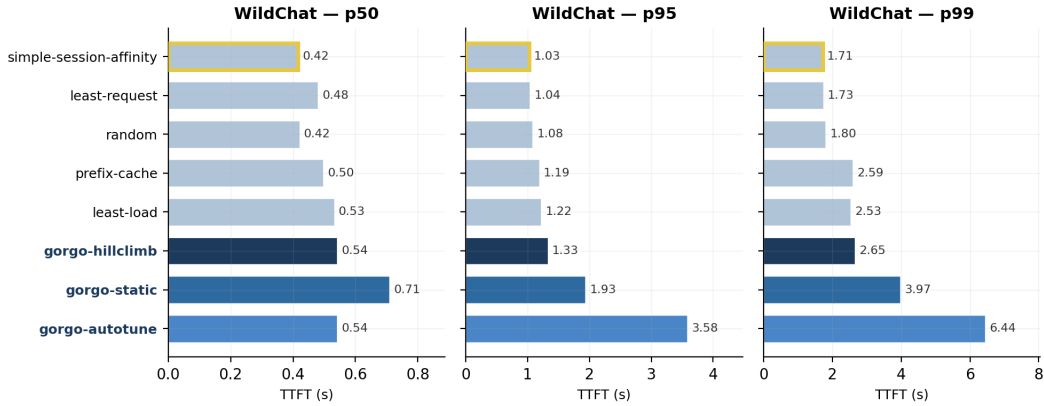


Figure 9: WildChat-4.8M replay, per-percentile TTFT (p50/p95/p99) across the eight policies. GORGO variants (dark blues) are not competitive: gorgo-static and gorgo-autotune are the slowest two policies at the tail, and gorgo-hillclimb sits mid-pack. The ranking inverts the ArtChat-411K result, consistent with the regime argument in §2.

483 The ranking is reversed: simple-session-affinity wins p95 and p99 by a large mar-
 484 gin, least-request is competitive, gorgo-hillclimb sits mid-pack, and gorgo-static and
 485 gorgo-autotune are the worst two policies. Two mechanisms drive this. First, the prefill term in
 486 Equation 2 is small at 2,925 tokens; the cost-model weights are mostly fitting noise. Second, intra-
 487 user reuse is 5.3% rather than 53%; even a perfect cache-routing decision saves only a few hundred
 488 tokens per request, well inside the cross-region RTT band, so chasing cache locality is counterpro-
 489 ductive when paired with a cross-region routing penalty. The gorgo-static weights frozen from
 490 an ArtChat-411K W1 are particularly mis-calibrated to this regime—a reminder that the cost-model
 491 parameters do not transfer across workload classes.

492 We include this result as a regime-sensitivity control, not as a contribution. The takeaway is the
 493 same one stated in §2: the gain from joint cache-network-queue routing depends on the workload
 494 having long prompts and substantial intra-user prefix reuse. Public chat datasets do not exercise this
 495 regime.

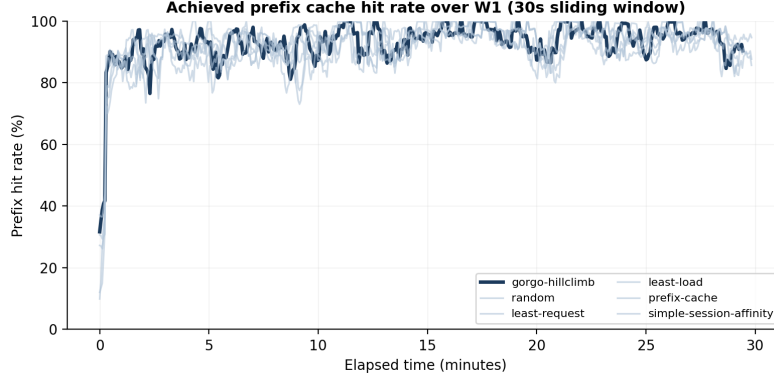


Figure 10: Achieved prefix hit rate over the W1 tuning window: the fraction of each request’s input tokens that were already cached on the replica the policy routed to (not the total cache available across all replicas). A higher rate means the routing decision exploited more of the available cache. Bold lines are EWMA-smoothed ($\alpha=0.06$); faint lines are 64-request rolling averages. gorgo-hillclimb converges from $\sim 45\%$ to $\sim 95\%$ within the first 5 minutes as the ES learns to route requests to their best-cached replica.

B GORGO Achieves Convergence of Cache Reuse

C Load-Weight Ablation: Single-Replica Concentration

When w_{load} is unconstrained (range $[10^{-5}, 5]$), the ES drives it to zero: the resulting policy routes 100% of traffic to a single replica (Figure 11). On the nighttime tuning trace (~ 1.2 req/s) this produces the best TTFT because every request hits a warm cache on the closest replica with no load penalty. On the heavier midday diurnal trace (~ 1.7 req/s, 47% more requests), the single replica saturates: decode throughput drops to 70 tok/s (vs. 119 tok/s with $w_{\text{load}}=6.38$) and E2E p95 inflates to 12.58 s— $4.8\times$ worse than least-request.

Constraining the ES search range to $w_{\text{load}} \in [0.1, 10.0]$ prevents this degenerate solution. The ES converges to $w_{\text{load}} \approx 6.38$, which actively avoids loaded replicas and achieves the best TTFT and E2E on both evaluation windows (§5.1). Table 10 compares the two configurations on the midday diurnal trace.

Table 10: Midday diurnal evaluation: unconstrained ($w_{\text{load}}=0$) vs. constrained ($w_{\text{load}}=6.38$) gorgo-static. Constraining w_{load} trades 3% TTFT p95 for $5\times$ better E2E p95.

	TTFT p50	TTFT p95	E2E p50	E2E p95	Decode	Concentration
$w_{\text{load}}=0$	0.190	1.101	2.07	12.58	70	100%
$w_{\text{load}}=6.38$	0.195	1.136	1.29	2.46	119	60%
Δ	+2.6%	+3.2%	−37.7%	−80.4%	+70%	

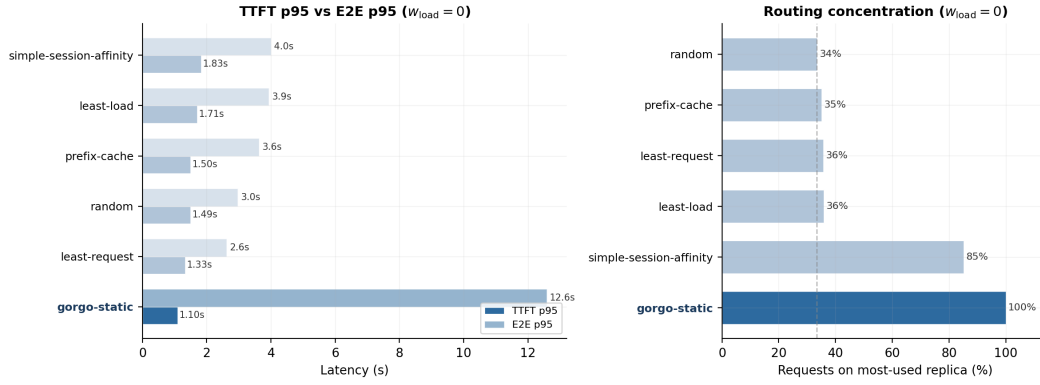


Figure 11: *Left*: TTFT p95 (dark) vs. E2E p95 (light) on the midday diurnal trace with $w_{load}=0$. *gorgo-static* wins TTFT but its E2E inflates to 12.6 s. *Right*: routing concentration. *gorgo-static* sends 100% of requests to one replica.